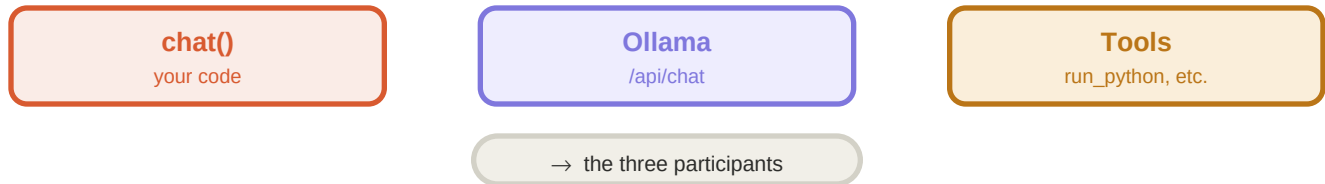

How tools get called and executed

A step-by-step walkthrough of the `chat()` function's tool-calling loop



The tool-calling loop involves three parties working together. **chat()** is your code — it manages the messages list and orchestrates the loop. **Ollama** is the LLM API — it receives messages and either responds with text (done) or requests tool calls (keep going). **Tools** are the real functions (`run_python`, `write_file`, etc.) that execute on your machine and return results.

The core loop is simple: send messages to Ollama, check if the response contains `tool_calls`. If yes, execute the tools, append results to messages, and resend. If no, the model is done — return the text. The messages list grows with each round, giving the model full context of everything that happened.

Message types

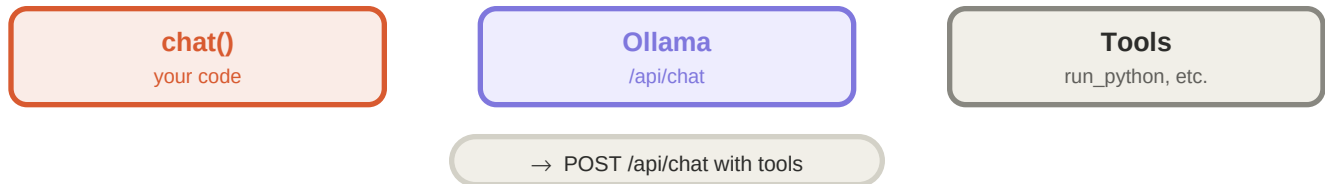
■ system ■ user ■ assistant ■ tool result ■ final text

The complete loop in pseudocode

```
for round in range(max_tool_rounds):
    response = send(messages + tools) to Ollama
    if response has no tool_calls:
        return response.content # DONE
    for each tool_call in response:
        result = execute(tool_call)
        messages.append(assistant_msg)
        messages.append(tool_result) # loop back and resend
```

The following pages walk through a concrete example: the executor agent writing a Flask app, discovering flask isn't installed, and fixing it.

Send initial request



What happens

The loop begins. `chat()` builds the payload containing the system prompt, user message, and the full tools array (all 5 tool schemas), then sends it to Ollama via POST request. This is round 0 of the for loop. The messages list starts with just two entries.

Code executing

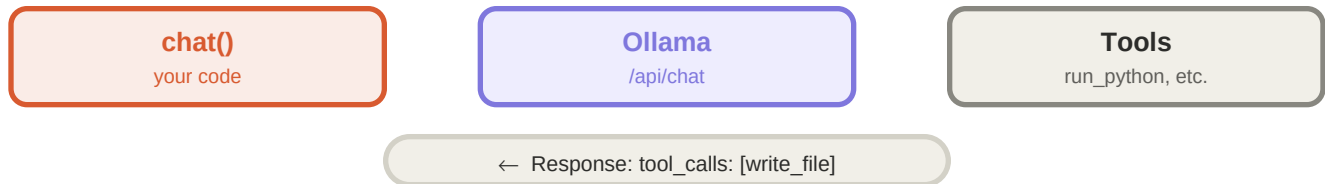
```
payload = { "model": model, "messages": messages, # [system, user] "tools": tools, # the 5
            tool schemas "stream": False } resp = requests.post(URL + "/api/chat", json=payload)
```

Messages list state



● New in this step: 2 messages added (green dot)

Model requests write_file



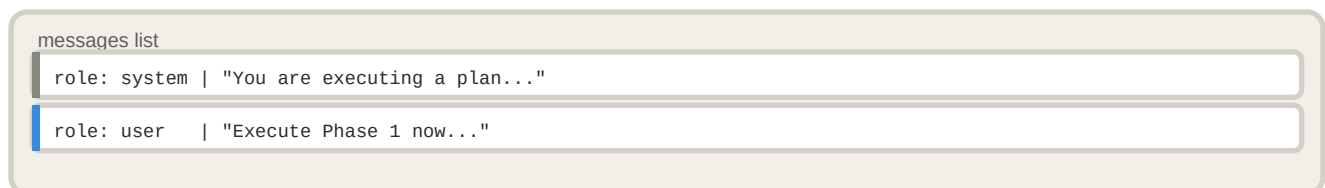
What happens

Ollama returns a message with `tool_calls` instead of plain text content. The model decided it needs to write a file before it can answer. The critical check happens: `tool_calls` is truthy, so the function does NOT return. Instead it falls through to the tool execution block below.

Code executing

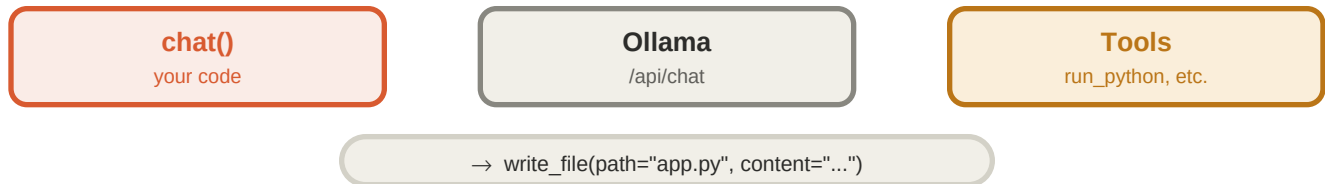
```
tool_calls = msg.get("tool_calls") # not empty! if not tool_calls: return msg["content"] #  
SKIPPED - tools requested # falls through to execute tools...
```

Messages list state



No new messages this step — response still being processed

Execute write_file



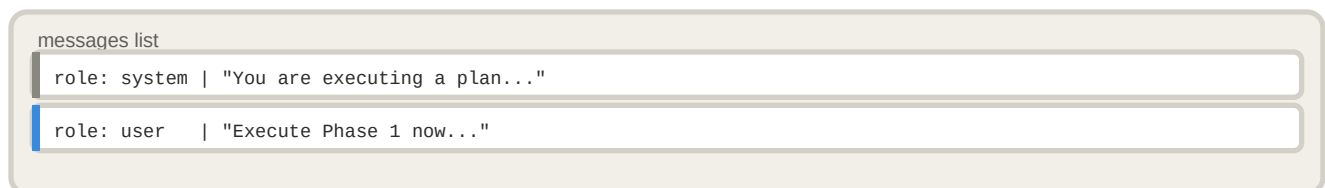
What happens

`chat()` looks up "write_file" in the `TOOL_FUNCTIONS` dictionary, which maps to the actual Python function. It unpacks the arguments the model provided using `**arguments` and calls the function. The file `app.py` gets written to disk as a real side effect — this is not a simulation.

Code executing

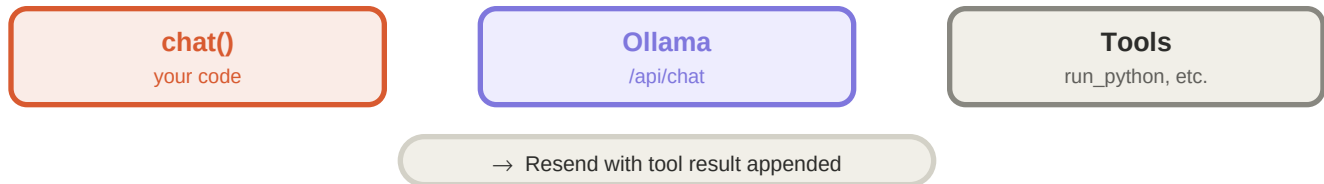
```
func = TOOL_FUNCTIONS["write_file"] # the real function
result = func( path="app.py",
content="from flask import Flask..." # 847 chars )
# result = "Wrote 847 chars to app.py"
```

Messages list state



No new messages this step — response still being processed

Append results and resend



What happens

Two messages get appended to the list: the assistant's original response (containing the `tool_calls` request), and the tool's result string. Then the entire messages list — now 4 entries — is sent back to Ollama. The model will see what it asked for and what it got back.

Code executing

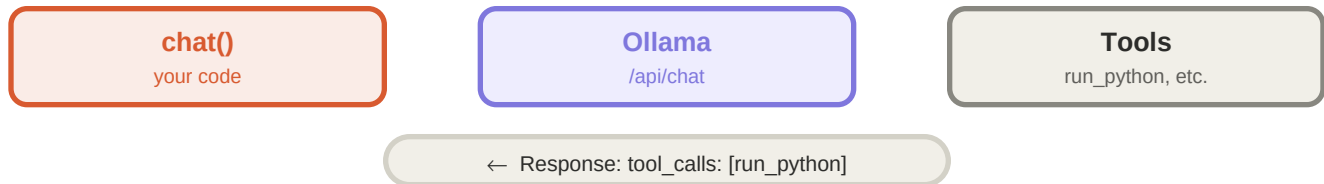
```
messages.append(msg) # assistant's tool_calls msg
messages.append({ "role": "tool",
                  "content": result # "Wrote 847 chars to app.py" }) # loop continues -> next iteration sends all 4 messages
```

Messages list state



● New in this step: 2 messages added (green dot)

Model requests run_python



What happens

Ollama sees the file was written successfully and now wants to test it. It responds with another `tool_calls` request — this time for `run_python`. Again, `tool_calls` is `truthy`, so the function does not return. The for loop continues into its next iteration.

Code executing

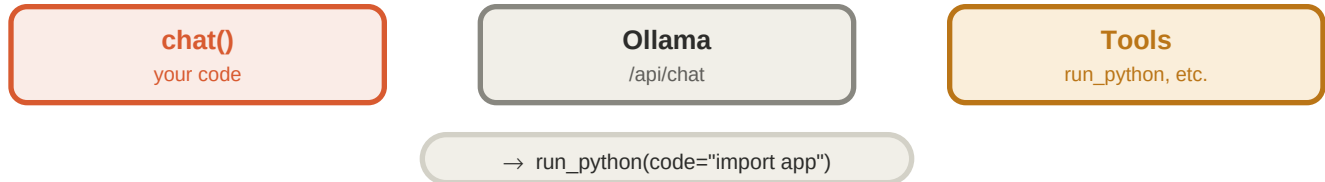
```
tool_calls = msg.get("tool_calls") # still not empty # model wants: run_python(code="import app; ...")
```

Messages list state



No new messages this step — response still being processed

Execute run_python — error



What happens

`chat()` executes `run_python`, which spawns a real subprocess running `python3 -c` with the code string. The subprocess fails with an `ImportError` — `flask` is not installed. The error output (`stderr` + exit code) becomes the tool result and gets appended to the messages list.

Code executing

```
result = subprocess.run( ["python3", "-c", code], capture_output=True, timeout=30 ) #  
stderr: "ModuleNotFoundError: No module named 'flask'" # exit code 1
```

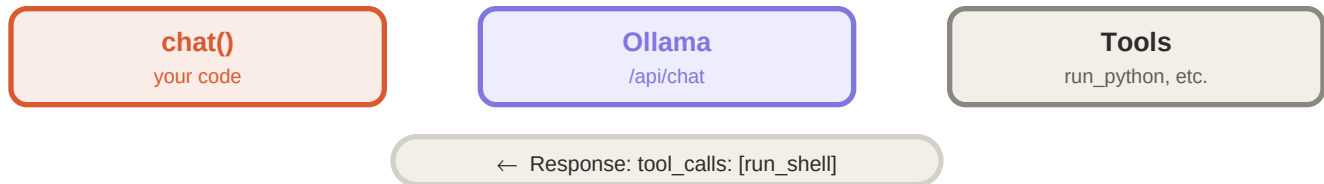
Messages list state

messages list

role: system	"You are executing a plan..."	
role: user	"Execute Phase 1 now..."	
role: assistant	tool_calls: [write_file(...)]	
role: tool	"Wrote 847 chars to app.py"	
role: assistant	tool_calls: [run_python(...)]	●
role: tool	"ModuleNotFoundError: flask [exit 1]"	●

● New in this step: 2 messages added (green dot)

Model self-corrects with run_shell



What happens

The model sees the ImportError in its conversation history and knows exactly what to do — install flask via pip. It requests run_shell. This is the self-correction loop in action: write → test → see error → fix → retest. You did not code this logic explicitly; the model figured out the fix from the error message.

Code executing

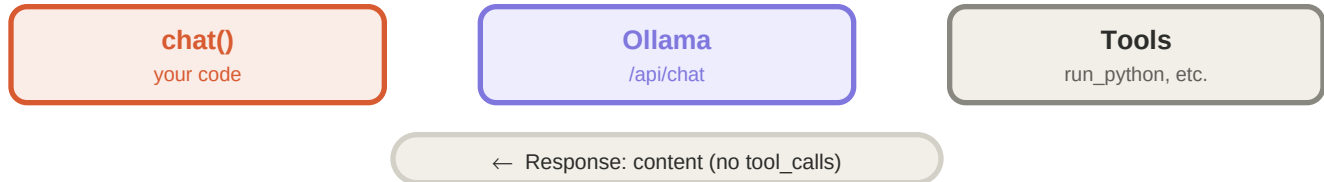
```
# model sees the error and self-corrects tool_calls = [{ "function": { "name": "run_shell",  
  "arguments": {"command": "pip install flask"} } }]
```

Messages list state

messages list	
role: system "You are executing a plan..."	
role: user "Execute Phase 1 now..."	
role: assistant tool_calls: [write_file(...)]	
role: tool "Wrote 847 chars to app.py"	
role: assistant tool_calls: [run_python(...)]	
role: tool "ModuleNotFoundError: flask [exit 1]"	
role: assistant tool_calls: [run_shell(...)]	●
role: tool "Successfully installed Flask-3.0.3"	●

● New in this step: 2 messages added (green dot)

Model returns final text



What happens

After installing flask and re-running the code successfully, the model finally responds with a text content field and no `tool_calls`. The if-check passes (not `tool_calls` is `True`), the function returns `msg["content"]`, and the for loop ends. The caller gets back the final summary string. The files are already on disk as side effects of the tool calls.

Code executing

```

tool_calls = msg.get("tool_calls") # None this time! if not tool_calls: return msg["content"]
# EXITS the loop # caller receives: "DONE: wrote app.py, installed flask..."
  
```

Messages list state

messages list

role: system	"You are executing a plan..."
role: user	"Execute Phase 1 now..."
role: assistant	tool_calls: [write_file(...)]
role: tool	"Wrote 847 chars to app.py"
role: assistant	tool_calls: [run_python(...)]
role: tool	"ModuleNotFoundError: flask [exit 1]"
role: assistant	tool_calls: [run_shell(...)]
role: tool	"Successfully installed Flask-3.0.3"
role: assistant	"DONE: wrote app.py, installed flask..."

● New in this step: 1 message added (green dot)

Key takeaways

The messages list is the memory

The LLM is stateless — it doesn't remember previous requests. The growing messages list IS the context. Every round sends the entire history, which is how the model knows it already wrote the file and saw the error.

One branch controls everything

The entire loop hinges on one check: if not tool_calls: return. When the model responds with tool_calls, the loop continues. When it responds with plain text, the loop ends. That's the only branching logic.

Self-correction is emergent, not coded

You never wrote "if ImportError then pip install." The model saw its own error in the conversation history and decided what to do. Your code provided the loop and the tools — the reasoning came from the model.

Side effects are real

When the model calls write_file, a real file appears on disk. When it calls run_python, a real subprocess executes. The tool results feed back into the conversation, but the side effects persist independently.

The safety valve prevents infinite loops

If the model keeps calling tools without ever giving a final text response, max_tool_rounds (default 15) caps the loop. The fallback sends one last request with tools=[] to force a text response.